

Implementing Basic Monitoring with Prometheus and Grafana

An Engineering Training Manual for Cluster Observability, Metric Scraping
Engine Configurations, and Dashboard Visualization

Document Identifier: K8S-OBS-2026

Version: 1.2.0 Observability Core Edition

Target Audience: Systems Engineering Interns, Reliability Specialists, Platform Operators

Prerequisites: Multi-Tier Dynamic Application Lifecycle Mapping Compliance

Date of Issuance: May 16, 2026

Classification: Internal Engineering Technical Resource

1. Introduction to the Core Pillars of Distributed Observability

1.1 Metrics, Logs, and Traces

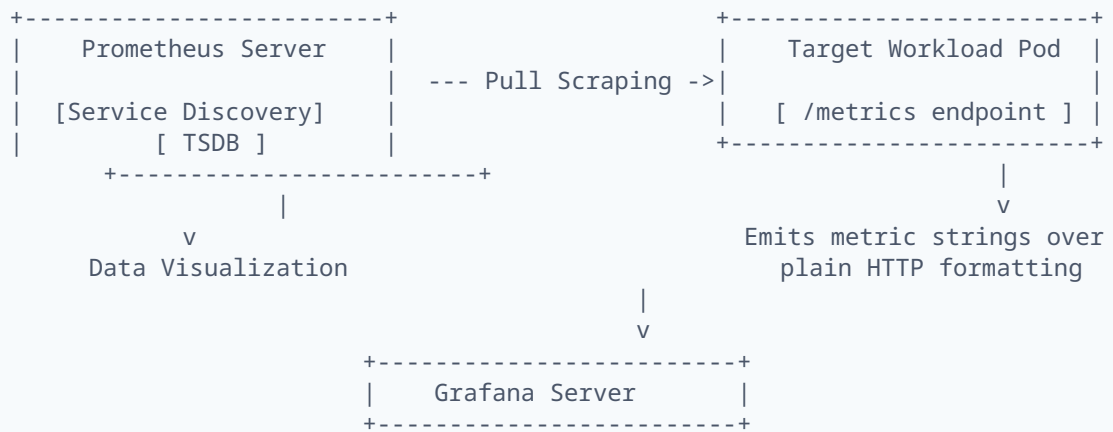
Managing a production environment demands deep visibility into system runtime contexts. In an ephemeral ecosystem like Kubernetes, debugging cannot happen by connecting to a physical host server instance manually. Instead, modern operators rely on the three pillars of observability:

- **Metrics:** Numeric, time-series data streams aggregated over uniform intervals. Metrics measure specific numeric trends, such as CPU consumption spikes, memory utilization ratios, request latencies, and transaction volumes. This manual is focused strictly on this pillar.
- **Logs:** Discrete, timestamped text strings emitted when internal code tracks specific execution paths. Logs are essential for retrospective deep-dives into application crash loops or syntax errors.
- **Traces:** End-to-end operational path data segments illustrating the structural lifespans of specific client requests as they cross multiple microservice boundaries.

1.2 The Prometheus Architectural Paradigm

Traditional monitoring tools often use a push-based model, where background daemons running inside application containers are responsible for continuously packaging and transmitting metrics out to an external monitoring endpoint.

Prometheus flips this paradigm entirely by implementing a highly efficient **Pull-Based Architecture**. Under this model, the centralized Prometheus Server is responsible for discovering targets and pulling metrics. Your applications do not push data; instead, they expose a plain-text HTTP endpoint (typically `/metrics`). Prometheus scans your cluster API server to locate active containers, visits these endpoints at regular intervals, and saves the data to its own high-performance Time Series Database (TSDB).



2. Prometheus Metric Typology and Plain-Text Exposition Format

2.1 The Four Prometheus Core Metric Types

To analyze cluster behaviors accurately, Prometheus categorizes metric telemetry streams into four distinct structural data types:

1. **Counter:** A cumulative metric value that can only increase over time or reset to zero on application restart. Used for tracking total counts of incoming web requests, transaction counts, or application error rates.
2. **Gauge:** A volatile numeric value that can swing up and down arbitrarily over time. Used to track current snapshot values like instantaneous memory allocation bytes, queue depths, or concurrent threads.
3. **Histogram:** Samples observations (typically things like request duration latencies or payload response sizes) and counts them in configurable, pre-allocated bucket boundaries. Provides detailed distribution insights.
4. **Summary:** Similar to a histogram, a summary calculates configurable quantiles (e.g., the 95th percentile) directly over a sliding window of time.

2.2 Deconstructing the OpenMetrics Text Format

The plain-text metric endpoint must conform to standard OpenMetrics formatting rules. Prometheus expects to read unstructured lines containing a metric string, key-value label metadata blocks, and the current value snapshot.

```
# HELP http_requests_total Total number of HTTP requests processed.
# TYPE http_requests_total counter
http_requests_total{method="POST",handler="/v1/charge",status="200"} 41052
http_requests_total{method="GET",handler="/healthz",status="500"} 12

# HELP node_memory_active_bytes Current volatile memory allocation.
# TYPE node_memory_active_bytes gauge
node_memory_active_bytes 8589934592
```

3. Blueprinting the Monitoring Architecture Namespace

3.1 Environmental Separation Principles

Observability infrastructure should always be deployed completely separate from core business logic workloads. This ensures that if an application namespace experiences a denial-of-service event, or if a severe memory leak impacts your application nodes, the monitoring framework remains online to gather and report diagnostic data.

3.2 The Infrastructure Namespace Manifest

Save the following declarative blueprint config as `monitoring-namespace.yaml` to establish an isolated workspace for our observability stack.

```
apiVersion: v1
kind: Container
apiVersion: v1
kind: Namespace
metadata:
  name: management-monitoring
  labels:
    tier: observability-infrastructure
    security-zone: management
```

4. Configuring Cluster Access Controls for Prometheus (RBAC)

4.1 Role-Based Access Control Privileges

To implement dynamic pull scraping, Prometheus needs to continually query the Kubernetes API server to discover the IP addresses of newly scheduled Pods, endpoints, and services.

Because the default cluster configuration blocks anonymous access to the API server, we must configure explicit Role-Based Access Control (RBAC) rules. This requires us to provision a cluster-wide ServiceAccount and bind it to a ClusterRole that grants read-only access to nodes, pods, services, and endpoints namespaces.

4.2 RBAC Configuration Blueprints

Save the following multi-object configuration layout as `prometheus-rbac.yaml` to configure access controls for Prometheus.

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: prometheus-service-account
  namespace: management-monitoring
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus-api-reader
rules:
  - apiGroups: [""]
    resources:
      - nodes
      - nodes/metrics
      - services
      - endpoints
      - pods
    verbs: ["get", "list", "watch"]
  - apiGroups: ["apps"]
    resources:
      - deployments
      - statefulsets
    verbs: ["get", "list", "watch"]
  - nonResourceURLs: ["/metrics"]
    verbs: ["get"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: prometheus-rbac-link
subjects:
  - kind: ServiceAccount
    name: prometheus-service-account
    namespace: management-monitoring
roleRef:
  kind: ClusterRole
  name: prometheus-api-reader
  apiGroup: rbac.authorization.k8s.io

```

5. Blueprinting the Core Prometheus Configuration Engine

5.1 Managing the Prometheus Corefile

The behavior of the Prometheus server is driven by a primary configuration file named `prometheus.yml`. This file establishes global scrape intervals and defines individual "jobs" that control how Prometheus interrogates your cluster infrastructure.

Instead of hardcoding individual Pod IPs into this file, we leverage Kubernetes **Service Discovery** configurations. The configuration below utilizes the `kubernetes_sd_mgs: pod`` directive, telling Prometheus to scan your cluster APIs continuously and automatically discover scraping targets based on labels and annotations.

5.2 The ConfigMap Manifest Configuration

Save the following configuration manifest as `prometheus-configmap.yaml`. This configures global scrape routines and includes structural rewrite rules to filter and scrape metrics from pods labeled with the annotation `prometheus.io/scrape: "true"`.


```

apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-engine-configuration
  namespace: management-monitoring
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s
      evaluation_interval: 15s

    scrape_configs:
      - job_name: 'kubernetes-pods'
        kubernetes_sd_configs:
          - role: pod
        relabel_configs:
          - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
            action: keep
            regex: true
          - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
            action: replace
            target_label: __metrics_path__
            regex: (.+)
          - source_labels: [__address__, __meta_kubernetes_pod_annotation_prometheus_io_port]
            action: replace
            regex: ([^:]+)(?::\d+)?;(\d+)
            replacement: $1:$2
            target_label: __address__
          - source_labels: [__meta_kubernetes_namespace]
            action: replace
            target_label: kubernetes_namespace
          - source_labels: [__meta_kubernetes_pod_name]
            action: replace
            target_label: kubernetes_pod_name

```

6. Deploying the Central Prometheus Server

6.1 Declarative Server Deployment

With our namespaces, access controls, and scrape configurations fully defined, we can deploy the core Prometheus server container. To ensure Prometheus has enough memory to buffer timeseries samples, we include explicit hardware resource requests and limits in the deployment manifest.

6.2 Comprehensive Server Deployment Manifest

Save the following declarative blueprint configuration layout as `prometheus-deployment.yaml`. This deployment attaches our dynamic ConfigMap directly into the container filesystem at path `/etc/prometheus/`, ensuring the engine boots with our customized scrapers.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: prometheus-core-deployment
  namespace: management-monitoring
  labels:
    app: prometheus-server
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prometheus-server
  template:
    metadata:
      labels:
        app: prometheus-server
    spec:
      serviceAccountName: prometheus-service-account
      containers:
        - name: prometheus-container
          image: prom/prometheus:v2.45.0
          args:
            - "--config.file=/etc/prometheus/prometheus.yml"
            - "--storage.tsdb.path=/prometheus/"
            - "--storage.tsdb.retention.time=15d"
          ports:
            - containerPort: 9090
              name: webui
          resources:
            requests:
              memory: "512Mi"
              cpu: "500m"
            limits:
              memory: "1Gi"
              cpu: "1000m"
          volumeMounts:
            - name: configuration-volume
              mountPath: /etc/prometheus/
            - name: temporary-storage
              mountPath: /prometheus/
      volumes:
        - name: configuration-volume
          configMap:
            name: prometheus-engine-configuration
        - name: temporary-storage
          emptyDir: {}

```

7. Blueprinting the Prometheus ClusterIP Interface Service

7.1 Establishing a Reliable Connection Target

To connect external visualization engines like Grafana to our time-series database, we must configure a stable connection target for Prometheus within the cluster management plane.

The manifest below sets up a standard **ClusterIP Service** that exposes the Prometheus dashboard and query APIs over port 9090 to other workloads running inside our management namespace.

7.2 The Prometheus Service Manifest

Save the following blueprint configuration as `prometheus-service.yaml` to finalize your core data layer deployment.

```
apiVersion: v1
kind: Service
metadata:
  name: prometheus-internal-service
  namespace: management-monitoring
  labels:
    app: prometheus-server
spec:
  type: ClusterIP
  ports:
    - port: 9090
      targetPort: webui
      protocol: TCP
  selector:
    app: prometheus-server
```

8. Deploying Grafana for Metric Dashboard Visualization

8.1 The Dashboard Component Layer

While Prometheus provides an incredibly powerful backend database and query engine, its built-in graphing user interface is intended primarily for basic diagnostic checks and raw metric troubleshooting.

To build rich, real-time telemetry dashboards for operations teams, we deploy **Grafana**. Grafana connects to Prometheus as a primary data source and allows developers to build rich, custom analytics dashboards that visualize the health of your workloads at a glance.

8.2 Comprehensive Grafana Deployment Manifest

Save the following combined configuration layout as `grafana-infrastructure.yaml`. This file deploys a dedicated Grafana instance and exposes its web interface to external users on port `3000` across your cluster nodes using a NodePort Service.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: grafana-deployment
  namespace: management-monitoring
  labels:
    app: grafana-ui
spec:
  replicas: 1
  selector:
    matchLabels:
      app: grafana-ui
  template:
    metadata:
      labels:
        app: grafana-ui
    spec:
      containers:
        - name: grafana-container
          image: grafana/grafana:10.0.0
          ports:
            - containerPort: 3000
              name: grafanaport
          env:
            - name: GF_SECURITY_ADMIN_PASSWORD
              value: "MoniSecured2026Pass"
          resources:
            requests:
              memory: "256Mi"
              cpu: "200m"
            limits:
              memory: "512Mi"
              cpu: "500m"
---
apiVersion: v1
kind: Service
metadata:
  name: grafana-edge-service
  namespace: management-monitoring
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: grafanaport
      nodePort: 30300
      protocol: TCP
  selector:
    app: grafana-ui

```

9. Instrumenting Application Workloads for Metric Scraping

9.1 Injecting Scrape Directives

Now that our observability stack is fully deployed, we need to instruct Prometheus to start pulling metrics from our application tiers. Thanks to the dynamic relabeling rules we wrote in our ConfigMap, we don't need to touch our Prometheus server configurations.

Instead, we simply inject standard **Annotations** directly into the template metadata of our application deployments. When the Prometheus service discovery engine scans the cluster APIs, it reads these annotations and automatically adds the corresponding container workloads to its active scraping loops.

9.2 Modifying the Application Template Blueprint

To enable automated metric collection on your application tier, update your application deployment manifests to include these exact structural metadata pointers:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-tier-deployment
  namespace: production
spec:
  replicas: 3
  template:
    metadata:
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/path: "/metrics"
        prometheus.io/port: "8080"
      labels:
        app: business-api
        tier: application
    # Remaining application container specs continue exactly as structured previously...
```

10. Step-by-Step Lab Execution and Verification Workflow

10.1 Order of Manifest Application

To initialize your observability lab cleanly and avoid dependency errors, apply the manifests to your cluster in this exact structural order:

```
# Step 1: Provision the isolated monitoring workspace namespace
kubectl apply -f monitoring-namespace.yaml

# Step 2: Establish API authorization roles for the service account
kubectl apply -f prometheus-rbac.yaml

# Step 3: Write the core configuration maps into cluster state
kubectl apply -f prometheus-configmap.yaml

# Step 4: Provision the Prometheus engine server and service hooks
kubectl apply -f prometheus-deployment.yaml
kubectl apply -f prometheus-service.yaml

# Step 5: Initialize the Grafana analytics and visualization interface
kubectl apply -f grafana-infrastructure.yaml
```

10.2 Connecting Grafana to the Prometheus Data Layer

Once all pods are running healthily, follow these verification steps to build your first live dashboard:

1. Open your browser and navigate to your Grafana edge web portal at `http://<WORKER-NODE-IP>:30300`.
2. Log in using the username `admin` and the password configured in your deployment manifest: `MoniSecured2026Pass`.
3. In the left-hand navigation menu, navigate to **Connections** and click on **Data Sources**.
4. Click **Add data source** and select **Prometheus**.
5. In the URL field, input the internal fully qualified domain name (FQDN) of your Prometheus service:

```
http://prometheus-internal-service.management-monitoring.svc.cluster.local:9090
```

6. Scroll down to the bottom of the page and click **Save & Test**. You should see a green confirmation banner stating *Data source is working*.

10.3 Formulating Your First Live Metric Query

With the data source linked, navigate to the **Explore** tab in Grafana to write your first Prometheus Query Language (**PromQL**) expressions.

To visualize the per-second rate of incoming HTTP connections across your application replicas over a sliding 5-minute window, enter the following expression:

```
rate(http_requests_total{kubernetes_namespace="production"}[5m])
```

To audit memory utilization trends across your container workloads, use this query to identify replicas that are approaching their configured limits:

```
container_memory_working_set_bytes{namespace="production", container!=""}
```